Alocação Ótima de Minisubestações em Sistemas de Distribuição Radiais

Fluxo de potência backward/forward + Simulated Annealing

Sistema-teste de 69 barras, 12,66 kV

Documentação detalhada do algoritmo, das equações e do código

Disciplina: Planejamento de Sistemas de Distribuição de Energia Elétrica

1. Objetivo e visão geral

O programa decide **em quais barras** de uma rede de distribuição devem ser instaladas N minisubestações (fontes locais de potência) de modo a **reduzir as perdas elétricas** e, consequentemente, o **custo total**. O usuário informa a quantidade de minisubestações, o preço de instalação de cada uma, a potência ativa (kW) e reativa (kvar) de cada uma e o preço por kW perdido. O programa devolve a lista de barras onde instalar e compara a rede antes e depois.

A solução segue a arquitetura **mestre-escravo** vista em aula (Aula 02): o algoritmo metaheurístico é o **problema mestre** (escolhe as posições) e o **fluxo de potência** é o **problema escravo** (calcula tensões, correntes e perdas de cada configuração candidata). O fluxo de potência é o "motor" do sistema.

1.1 Por que uma heurística?

Com N minisubestações e 68 barras candidatas existem C(68, N) combinações. Para N = 1 são 68; para N = 3 são 50.116; para N = 6 já passam de 100 milhões. Cada combinação exige um fluxo de potência completo, então a busca exaustiva deixa de ser viável rapidamente. Uma metaheurística encontra soluções muito boas visitando só uma pequena fração do espaço de busca.

1.2 Estrutura mestre-escravo

Papel	Componente	Função
Mestre	Simulated Annealing (otimiza_minisubs.py)	Propõe conjuntos de barras e decide quais aceitar
Escravo	Fluxo backward/forward (fluxo_de_potencia.py)	Para cada conjunto, calcula V, I, perdas
Função objetivo	avaliar()	Converte perdas + instalação em custo (R$)

2. Arquivos do projeto

Arquivo	Responsabilidade
main.py	Programa principal: orquestra leitura, ordenação, caso base, entrada do usuário, otimização e relatório.
ler_dados.py	Lê o arquivo do sistema (barras, linhas, tensão) e devolve um dicionário.
ordena.py	Ordena as linhas por busca em largura (BFS) a partir da subestação, garantindo a ordem montante-jusante.
fluxo_de_potencia.py	Varredura backward/forward. Recebeu o parâmetro opcional *injecoes* para modelar as minisubestações.
otimiza_minisubs.py	Entrada de parâmetros, função objetivo, Simulated Annealing e relatório de onde instalar.
069BusSystem.txt	Dados do sistema de 69 barras: 12,66 kV, 69 barras, 68 linhas.

2.1 Sequência de execução (main.py)

- Define o caminho do arquivo (argumento, pasta do script ou pasta Downloads) e lê os dados.
- Ordena as linhas a partir da SE (ordena_linhas).
- Roda o fluxo de potência do **caso base**, sem minisubestações, e imprime o resultado.
- Pergunta ao usuário os 5 parâmetros (quantidade, preço de instalação, kW, kvar, preço por kW perdido).
- Executa o Simulated Annealing (5 reinícios independentes).
- Imprime **onde instalar**, a comparação antes x depois e o fluxo de potência final.

3. Leitura e ordenação da rede

3.1 Leitura dos dados

O arquivo tem três blocos: dados gerais (número de barras, número de linhas, tensão em kV), *Bars_data* (id, tipo SE/PQ, carga ativa SaReal em kW, carga reativa SaImag em kvar) e *Lines_Data* (id, barra de origem Ni, barra de destino Nj, resistência ZaaReal e reatância ZaaImag em ohms). A função *ler_arquivo* percorre o texto linha a linha, identifica cada bloco pelo título e converte os valores para int/float.

3.2 Ordenação por busca em largura (BFS)

O método backward/forward exige que cada linha seja processada depois da linha que a alimenta (para a varredura direta) e antes dela (para a varredura inversa). A função *ordenar_linhas* faz isso: começa na barra SE, coloca em uma fila todas as linhas que saem do nó atual e avança em camadas. O resultado são três listas paralelas (Ni_ordenado, Nj_ordenado, id_ordenado). Como os filhos de um nó sempre aparecem depois dele, percorrer a lista de trás para frente visita as folhas primeiro, e de frente para trás visita a SE primeiro.

Se alguma linha não for alcançada a partir da SE (rede desconectada ou com laço), o programa emite um aviso.

4. Fluxo de potência backward/forward

Redes de distribuição são **radiais** e têm relação X/R alta ou moderada, o que torna o Newton-Raphson clássico menos eficiente. O método da **soma de correntes** (Shirmohammadi et al.) explora a topologia em árvore e converge rapidamente. Modelo adotado: fase equivalente monofásica, carga de potência constante, sistema em por unidade (pu).

4.1 Valores de base

$$Z_{base} = kV^2 / MVA \qquad I_{base} = kVA / (raiz(3) \times kV)$$

Com S_{base} = 10 MVA e V_{base} = 12,66 kV, resulta Z_{base} = 16,03 ohm. Cargas e impedâncias são divididas pelas respectivas bases: S(pu) = (kW + j kvar) / S_{base} e Z(pu) = Z(ohm) / Z_{base}.

4.2 Algoritmo iterativo

A tensão de todas as barras começa em 1,0 pu (*flat start*). A cada iteração:

Passo 1 - Corrente nodal. Para cada barra i (exceto a SE):

$$I_i = (S_i / V_i)^*$$

em que * indica conjugado complexo e S_i é a potência *líquida* consumida na barra.

Passo 2 - Backward (folhas para a SE). A corrente de cada linha é a corrente da carga da barra de destino mais as correntes de todas as linhas a jusante:

$$J_{ij} = I_j + \text{soma das correntes J das linhas que saem de j}$$

No código, o dicionário *soma_jusante* acumula essa soma enquanto a lista ordenada é percorrida de trás para frente.

Passo 3 - Forward (SE para as folhas). Com as correntes fixas, as tensões são atualizadas:

$$V_j = V_i - Z_{ij} \times J_{ij}$$

Isto é a Lei de Ohm da aula: $V_i - V_j = Z_{ij} \times I_{ij}$, com Z = R + jX. A tensão da SE é mantida fixa (barra de referência, ângulo zero).

Passo 4 - Critério de convergência. Calcula-se o erro de potência em cada barra:

$$dS_i = V_i \times (I_i)^* - S_i$$

O processo termina quando as partes real e imaginária de todos os dS_i são menores ou iguais à tolerância (padrão 1e-5 pu) ou quando atinge 100 iterações.

4.3 Pós-processamento

$$\text{Perdas} = \text{soma sobre as linhas de } Z_l \times |J_l|^2 \times S_{base}$$

O resultado é dado em kW (parte real) e kvar (parte imaginária). A potência total fornecida pela SE é a soma de $V_{SE} \times J^*$ das linhas que saem dela. O código também informa tensões em pu, graus e kV, correntes em pu e ampères e a barra de menor tensão. Pela norma de operação citada na aula, a faixa adequada de tensão é de 0,95 a 1,05 pu.

5. Modelagem das minisubestações

Cada minisubestação é modelada como uma **injeção constante de potência** $S_{mini} = P + jQ$ na barra onde for instalada. Do ponto de vista da rede, ela reduz a carga líquida vista naquela barra:

$$S_{liquida,i} = S_{carga,i} - S_{mini,i}$$

No código isso é feito com o novo parâmetro *injecoes* de *fluxo_de_potencia()*, um dicionário {barra: complex(kW, kvar)}. Logo após montar S de cada barra, o código subtrai S_{mini}/S_{base} das barras escolhidas. Sem o parâmetro, o fluxo se comporta como antes.

Se a injeção for maior que a carga da barra, a corrente nodal muda de sinal e a barra passa a exportar potência para a rede, o que é permitido pelo modelo. Em cada barra pode ser instalada no máximo uma minisubestação.

6. Função objetivo

$$\text{Custo} = N \times C_{inst} + C_{kW} \times \text{Perdas}_{kW} + \text{Penalidade de tensão}$$

Os termos têm o seguinte significado:

- **N x C_{inst}:** custo de instalação. Como o usuário fixa N, este termo é constante durante a busca.
- **C_{kW} x $Perdas_{kW}$:** custo das perdas ativas, com o preço por kW informado. Este é o termo que o algoritmo realmente reduz.
- **Penalidade de tensão:** soma, sobre todas as barras, de 1e6 x max(C_{kW}, 1) x violação, em que a violação é a distância (em pu) da tensão até a faixa de 0,95 a 1,05. Soluções que deixam a tensão fora da faixa ficam muito caras e são evitadas. Para desativar, use v_min=None.
- **Inviabilidade:** se o fluxo de potência não convergir, soma-se 1e12 ao custo, descartando a solução.

Minimizar perdas e minimizar custo coincidem neste problema porque o custo de instalação é fixo; se o número de unidades fosse variável, haveria um compromisso real entre investir mais e perder menos.

7. O algoritmo Simulated Annealing

O Simulated Annealing (recozimento simulado) é inspirado no resfriamento lento de metais. A "temperatura" T controla quanto o algoritmo aceita soluções piores: no início aceita muitas, o que permite explorar o espaço e escapar de mínimos locais; ao final, aceita quase nenhuma e refina a melhor solução encontrada.

7.1 Representação e vizinhança

Uma solução é uma lista de N barras distintas, escolhidas entre as 68 candidatas (a SE é excluída). Um **vizinho** é obtido trocando **uma** das barras escolhidas, sorteada ao acaso, por outra barra livre, também sorteada.

7.2 Critério de aceitação (Metropolis)

delta = Custo(vizinho) - Custo(atual) Aceita se delta < 0, ou com probabilidade exp(-delta / T)

Melhorias são sempre aceitas. Pioras são aceitas com probabilidade que cai quando delta cresce ou T diminui. A melhor solução já vista é guardada separadamente, de modo que uma piora aceita nunca a perde.

7.3 Parâmetros

Parâmetro	Valor padrão	Papel
T0 (temperatura inicial)	10% do custo da solução inicial	No início, uma piora de cerca de 10% é aceita com probabilidade de aproximadamente 37%
alfa (resfriamento)	0,95	T = 0,95 x T a cada patamar (resfriamento geométrico)
iter_por_temp	20	Vizinhos testados em cada temperatura
T_final	1e-6 x custo inicial	Critério de parada
reinícios	5 (sementes 0 a 4)	Execuções independentes; fica a melhor

Com esses valores, cada execução percorre cerca de 225 patamares de temperatura, ou aproximadamente 4.500 avaliações de vizinhos. Como muitas combinações se repetem, as avaliações reais de fluxo de potência são bem menos.

7.4 Cache de avaliações

O resultado de cada configuração é guardado em um dicionário cuja chave é o *frozenset* das barras. Assim a ordem das barras não importa e uma configuração já avaliada não recalcula o fluxo de potência. Isso acelera bastante a busca, sobretudo nas temperaturas baixas, em que o algoritmo revisita as mesmas regiões.

7.5 Pseudocódigo

```
para r em 0..4:                          # reinicios
    atual  <- N barras aleatorias
    custo  <- avaliar(atual)             # roda o fluxo de potencia
    melhor <- atual
    T <- 0.1 * custo
    enquanto T > T_final:
        repita 20 vezes:
            viz <- atual com 1 barra trocada por uma livre
            delta <- avaliar(viz) - custo
            se delta < 0  ou  random() < exp(-delta/T):
                atual <- viz ; custo <- avaliar(viz)
                se custo < custo(melhor): melhor <- atual
        T <- 0.95 * T
devolve a melhor solucao entre os 5 reinicios
```

8. Resultados no sistema de 69 barras

Caso base (sem minisubestações): perdas ativas de **224,97 kW**, perdas reativas de 102,06 kvar e menor tensão de 0,909 pu, abaixo do limite de 0,95 pu. A maior carga do sistema está na barra 61 (1244 kW + j888 kvar).

8.1 Teste 1: validação com N = 1

Minisubestação de 1000 kW + j600 kvar, R$ 50.000 de instalação e R$ 1.000 por kW perdido. O algoritmo escolheu a barra 61, com perdas de 65,82 kW (redução de 70,7%). A **busca exaustiva** nas 68 barras confirmou que a barra 61 é a melhor (a segunda melhor, a barra 62, resulta em 66,14 kW). Isso valida o funcionamento do algoritmo.

8.2 Teste 2: N = 3

Três minisubestações de 500 kW + j300 kvar, R$ 50.000 cada e R$ 1.000 por kW perdido:

Barra	P (kW)	Q (kvar)	|V| antes (pu)	|V| depois (pu)
61	500	300	0,91237	0,98274
62	500	300	0,91208	0,98328
64	500	300	0,90979	0,98471

Indicador	Antes	Depois
Perdas ativas	224,967 kW	32,387 kW (-85,6%)

Indicador	Antes	Depois
Perdas reativas	102,062 kvar	19,082 kvar
Menor tensão	0,90922 pu	0,96928 pu
Custo das perdas	R$ 224.967,02	R$ 32.386,56
Custo de instalação	-	R$ 150.000,00
Custo total	R$ 224.967,02	R$ 182.386,56

As barras escolhidas ficam no final do alimentador, onde está a maior carga: a injeção local reduz a corrente que percorre o tronco, e as perdas são proporcionais ao **quadrado** da corrente. Além disso, todas as tensões passam a respeitar a faixa de 0,95 a 1,05 pu.

9. Como executar

```
python main.py                      # usa 069BusSystem.txt da pasta do script
python main.py caminho/arquivo.txt  # ou informe o caminho do arquivo
```

Em seguida, responda às cinco perguntas. Exemplo de entrada:

```
Quantas minisubestacoes deseja inserir? 3
Preco de instalacao de CADA uma (R$): 50000
Potencia ativa de cada uma (kW): 500
Potencia reativa de cada uma (kvar): 300
Preco por kW perdido (R$/kW): 1000
```

A saída principal é o bloco **ONDE INSTALAR AS MINISUBESTAÇÕES**, com a lista de barras (>>> Instalar nas barras: [61, 62, 64]), seguido da comparação antes x depois e do fluxo de potência completo com as minisubestações.

10. Limitações e possíveis extensões

10.1 Hipóteses e limitações

- A rede é modelada como **monofásica equivalente** (trifásica equilibrada), carga de potência constante e uma única condição de carga (sem curva diária).
- A minisubestação injeta potência fixa (P + jQ) informada pelo usuário; não há limite de capacidade da rede nem controle de tensão da unidade.
- O preço por kW perdido é um valor único. Se for um custo anual, multiplique pelo horizonte de tempo (por exemplo, horas por ano x tarifa) antes de digitar.
- O número de unidades é fixo, e a posição é restrita às barras existentes (uma unidade por barra).
- Por ser heurístico, o resultado **não tem garantia de ótimo global**; os reinícios e o cache aumentam a confiabilidade. Para N pequeno, a busca exaustiva serve como conferência.

10.2 Extensões sugeridas

- Tornar N variável (de 0 a N máximo), fazendo o custo de instalação participar da decisão.
- Dimensionar também P e Q de cada unidade como variáveis de decisão.

- Incluir curva de carga horária e custo de energia por patamar.
- Comparar com outras metaheurísticas (algoritmo genético, PSO, busca tabu) usando o mesmo fluxo de potência como problema escravo.
- Adicionar restrições de corrente máxima nos condutores.